

UNIVERSIDAD MILITAR NUEVA GRANADA



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

Guías de Prácticas de Laboratorio	Identificación: GL-AA-F-1	
	Número de Páginas: 10	Revisión No.: 2
	Fecha Emisión: 2018/01/31	
Laboratorio de: Programación 3		
Título de la Práctica de Laboratorio: Estructuras de datos y Algoritmos de ordenamiento		

Elaborado por: Ing. Ricardo Castillo, Ph.D. Docente tiempo completo Ing. Nathaly Sánchez, M.Sc. Docente hora cátedra	Revisado por: Ing. Luís Francisco Niño, M.Sc Jefe de área robótica	Aprobado por: Ing. Darío Amaya, Ph.D. Director de Programa
---	---	---



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

Control de Cambios

Descripción del Cambio	Justificación del Cambio	Fecha de Elaboración / Actualización
Actualización del procedimiento	La temática no cambia, se ajusta el entregable para evitar plagios	2024/01/22
Actualización del procedimiento	La temática no cambia, se ajusta el entregable para evitar plagios	2024/07/12
Actualización del procedimiento	La temática no cambia, se ajusta el entregable para evitar plagios	2025/17/01

UNIVERSIDAD MILITAR NUEVA GRANADA



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

1. **FACULTAD O UNIDAD ACADÉMICA:** Ingeniería

2. **PROGRAMA:** Ingeniería Mecatrónica

3. **ASIGNATURA:** Programación 3

4. **SEMESTRE:** III

5. **OBJETIVOS:**

General: Diferenciar e implementar diferentes métodos de ordenamiento

Específicos:

- Utilizar programación orientada a objetos junto con listas y matrices en Python
- Utilizar algoritmos de ordenamiento con arreglos
- Comparar los algoritmos a nivel de complejidad

6. **MATERIALES, REACTIVOS, INSTRUMENTOS, SOFTWARE, HARDWARE O EQUIPOS DEL LABORATORIO:**

DESCRIPCIÓN (<i>Material, reactivo, instrumento, software, hardware, equipo</i>)	CANTIDAD	UNIDAD DE MEDIDA
NA		

7. **MATERIALES, REACTIVOS, INSTRUMENTOS, SOFTWARE, HARDWARE O EQUIPOS DEL ESTUDIANTE:**

DESCRIPCIÓN (<i>Material, reactivo, instrumento, software, hardware, equipo</i>)	CANTIDAD	UNIDAD DE MEDIDA
Computador	1	NA
Python 3.13.5	1	NA
Software IDE (Microsoft Visual Studio Code)	1	NA

8. **PRECAUCIONES CON LOS MATERIALES, REACTIVOS, INSTRUMENTOS Y EQUIPOS A UTILIZAR:**

Utilizar adecuadamente el equipo de cómputo para realizar la práctica de laboratorio.



9. PROCEDIMIENTO, MÉTODO O ACTIVIDADES:

1. Arreglos Estáticos en Python

Un arreglo o array es una secuencia de datos del mismo tipo. Los datos se llaman elementos del array y se enumeran consecutivamente 0,1,2,3,..., estos valores se denominan valores índice o subíndice del array tal como en matemáticas $\langle a_0, a_1, a_2, a_3, \dots \rangle$. El tipo de elementos almacenados en el array puede ser cualquier dato simple

En Python, no existen arreglos estáticos como en Java, pero se pueden usar listas (list), que son estructuras dinámicas que cumplen funciones similares.

Creación de un array (como lista) en Python

```
a = [0] * 5          # Lista de 5 enteros con valor 0
b = [0.0] * 10       # Lista de 10 flotantes con valor 0.0
```

Forma general:

```
nombre_lista = [valor_inicial] * numero_de_elementos
```

Asignación de valores:

```
c = [''] * 10
c[0] = 'z' # Asigna 'z' a la primera posición
```

```
t = [0.0] * 5
t[3] = -1.203 # Asigna -1.203 a la cuarta posición
```

```
tabla = [1, 6, 3, 4, 8] # Creación y asignación de valores simultáneamente
```

Tamaño de un array:

```
v = [0.0] * 15
print(len(v)) # Imprime 15, número de elementos de v
```

Arreglos multidimensionales:

Python permite listas dentro de listas para simular matrices (arreglos multidimensionales):

```
# Matriz de 80 filas y 24 columnas (inicializada en 0)
posiciones = [[0 for _ in range(24)] for _ in range(80)]
```

```
# Matriz con valores definidos
tabla = [
```



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

```
[1, 6, 3, 4, 8],  
[2, 4, 6, 7, 9]  
]
```

```
tabla2 = [  
    [1, 6, 3, 4, 8],  
    [2, 4, 6, 7, 9],  
    [1, 2, 3, 4, 5]  
]
```

Y en forma general para arreglos bidimensionales:

```
filas = 3  
columnas = 5  
nombre_array = [[0] * columnas for _ in range(filas)]
```

Ejemplo de arreglo tridimensional:

```
PAGINAS = 500  
LINEAS = 45  
COLUMNAS = 80  
  
# Creación del arreglo 3D  
libro = [[['' for _ in range(COLUMNAS)] for _ in range(LINEAS)] for _  
in range(PAGINAS)]  
  
# Ejemplo de procesamiento  
for pagina in range(PAGINAS):  
    for linea in range(LINEAS):  
        for columna in range(COLUMNAS):  
            # procesar libro[pagina][linea][columna]  
            pass
```

Cadenas

Una cadena es una secuencia de caracteres delimitada entre comillas dobles. “AMIGOS” es una cadena compuesta de 6 elementos. En Python, las cadenas (str) son objetos, al igual que en Java.

```
nombre = "Juan"  
vacía = ""  
rotulo = "\n\tLista de pasajeros\n=====
```

Métodos de Cadenas:



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

```
# length: tamaño de la cadena
programa = "Mecatrónica"
tamaño = len(programa) # tamaño = 11

# concat: concatenación
c1 = "Hola"
c2 = "Mundo"
c1 = c1 + c2 # c1 = "HolaMundo"

# substring: subcadena
c = "Mecatrónica"
d = c[4:8] # d = "tron" (inicio incluido, fin excluido)

# equals: comparación
c = "Mecatrónica"
d = "Mecatronics"
b = (c == d) # b = False
```

2. Algoritmos de ordenamiento

En ciencias de la computación, un algoritmo de ordenamiento es aquel que pone los elementos de una lista en cierto orden. Los órdenes más utilizados son el orden numérico y el orden lexicográfico.

Entrada: Una secuencia de n números $\langle a_0, a_1, a_2, a_3, \dots, a_n \rangle$

Salida: Una permutación (reordenamiento) $\langle a'_0, a'_1, a'_2, a'_3, \dots, a'_n \rangle$ de la secuencia de entrada tal que $a'_0 \leq a'_1 \leq a'_2 \leq a'_3 \leq \dots \leq a'_n$

Método de burbuja:

El ordenamiento de burbuja (bubble-sort) es el algoritmo de clasificación más simple que funciona intercambiando repetidamente los elementos adyacentes si están en el orden incorrecto.

```
BUBBLE-SORT(A)
1 for j ← 1 to length[A] - 1
2   for i ← 1 to length[A] - j - 1
3     if A[i] > A[i+1]
4       ▶ Intercambie A[i] y A[i+1]
```

Método de inserción:

El ordenamiento por inserción (insertion-sort) funciona de la misma manera



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento como muchas personas clasifican una baraja de cartas. Comenzamos con la mano izquierda vacía y las cartas boca abajo sobre la mesa. Luego retiramos una tarjeta a la vez de la mesa y la insertamos en la posición correcta en la mano izquierda. Para encontrar la posición correcta de una carta, la comparamos con cada una de las cartas que ya están en la mano, de derecha a izquierda.

```

INSERTION-SORT(A)
1  for j ← 2 to length[A]
2    do key ← A[j]
3      ▶ Inserte A[j] en la secuencia ordenada A[1 ... j - 1].
4      i ← j - 1
5      while i > 0 and A[i] > key
6        do A[i + 1] ← A[i]
7          i ← i - 1
8      A[i + 1] ← key
    
```

Método de selección:

La ordenación por selección (selection-sort) es un algoritmo que selecciona el elemento más pequeño de una lista sin clasificar en cada iteración y coloca ese elemento al principio de la lista sin clasificar.

```

SELECTION-SORT(A)
1  for i ← 0 to length[A] - 2
2    min ← i
2    for j ← i+1 to length[A] - 1
3      if A[j] < A[min]
4        min ← j
5    ▶ Intercambie A[i] y A[min]
    
```

Método merge:

La ordenación por mezcla (merge-sort) utiliza la estrategia divide y vencerás. Divide el arreglo de entrada en dos mitades, se llama a sí mismo para las dos mitades y luego fusiona las dos mitades ordenadas. La función merge() se usa para fusionar dos mitades. La función merge(A, l, m, r) asume que A[l .. m] y A[m + 1 .. r] están ordenados y fusiona los dos subarreglos ordenados en uno.

```

mergesort(A, l, r)
1  if r > l
    // Encuentre el punto medio para dividir el arreglo en dos mitades:
2    mitad m = l + (r-l)/2
    // Llame mergesort para la primera mitad:
    
```



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

```

3      mergesort(A, l, m)
      // Llame mergesort para la segunda mitad:
4      mergesort(A, m+1, r)
      // Mezcle las dos mitades ordenadas en 3 y 4:
5      merge(A, l, m, r)

mergesort(A, l, m, r)
    //crear un arreglo temporal
1  temp[r - l + 1]
2  i ← l; j ← m + 1; k ← 0 //índices para ambos intervalos
    // recorrer ambos arreglos y en cada iteración agregar el menor de
    //ambos elementos a temp
3  while i < m and j < r
4      if A[i] <= A[j]
5          temp[k] ← A[i]
6          k ← k + 1; i ← i + 1
7      else
8          temp[k] ← A[j]
9          k ← k + 1; i ← i + 1
    //agregar elementos a la izquierda del primer intervalo
10 while i < m
11     temp[k] ← A[i]
12     k ← k + 1; i ← i + 1
    //agregar elementos a la izquierda del segundo intervalo
13 while j < r
14     temp[k] ← A[j]
15     k ← k + 1; i ← i + 1
    //copiar temp al interval original
16 for i ← l to r
17     A[i] ← temp[i - l]
```

3. Ejercicios aplicados

3.1 Implemente un programa de consola que permita obtener las siguientes operaciones matemáticas a partir de arreglos y/o números escalares ingresados por el usuario:

- Suma de matrices
- producto de matrices
- Inversa de una matriz
- Producto de una matriz por un vector

Implementar estas operaciones dentro de una clase, en un archivo aparte del *main*, utilizar funciones *get* para retornar los resultados al *main*. La clase implementada debe tener constructor. Todas las operaciones deben implementarse utilizando listas como matrices, **no utilizar la librería numpy para programarlas**. La inversa de una matriz debe funcionar para matrices



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento
cuadradas de cualquier tamaño

3.2 Escriba un programa que ordene una lista de **números flotantes aleatorios** por los métodos *burbuja*, *inserción*, *selección* y *mergesort*. utilizar funciones *get* para retornar los resultados al *main*. La cantidad de números de la lista será pedida al usuario en el *main*. Realizar también el ordenamiento utilizando el método *sort* de Python.

Implementar cada una de estas operaciones dentro de una única clase en el *main*. La clase implementada debe tener constructor.

4. RESULTADOS ESPERADOS:

- Para los puntos 3.1, 3.2 los estudiantes deben entregar un código funcional el cual contenga **un menú** en donde se seleccionen las opciones dependiendo del punto a desarrollar. **El programa no se debe cerrar al momento de finalizar una de las opciones, sino que debe dar la opción de poder continuar con la siguiente.**
- En el punto 3.1 y 3.2 los estudiantes deben diseñar un **submenú** que permita seleccionar la opción del respectivo punto, para el punto 3.2
- Desarrollar cada uno de los ejercicios utilizando programación orientada a objetos **con una sola clase por archivo, también debe existir un archivo para el main**
- En el main se deben llamar las funciones por medio de objetos (no utilizar llamados estáticos).
- Cada una de las clases debe contener un método constructor que inicialice los datos miembro de esta, los métodos dentro de la clase no deben recibir parámetros.
- Los métodos dentro de la clase no deben retornar, se implementará un método *get* para retornar los resultados al main

5. CRITERIO DE EVALUACIÓN A LA PRESENTE PRÁCTICA:

El estudiante debe entregar el programa funcionando, su código fuente y sustentar su funcionamiento.

Criterio	No cumple	Cumple parcialmente	Cumple totalmente

UNIVERSIDAD MILITAR NUEVA GRANADA



Laboratorio 2. Estructuras de datos y Algoritmos de ordenamiento

Sustenta satisfactoriamente y responde a las preguntas del docente	0	0.7	1.4
Implementa los algoritmos solicitados	0	1.5	3
Presenta claridad y orden y el código desarrollado es orientado a objetos	0	0.3	0.6
Total			5.0

6. REFERENCIAS

<https://docs.python.org/es/3.13/tutorial/datastructures.html>

<https://numpy.org/doc/stable/reference/generated/numpy.matrix.html>